

Introduce yourself in the chat!

Tell us your program and concentration

Hello there



I'm Omer (the batter on the left), your instructor for today!

- Graduated from the MI program at the iSchool where I studied Data Science
- Working as a data scientist for the Ontario Public Service
- And I love teaching!

Housekeeping

- This session is interactive! Please ask questions when prompted, I'll be stopping regularly
- We will have a short break in the middle (stretch, grab some tea)
- In case, we aren't able to cover something properly, we will skip it. The goal is to learn the content, not rush through
- We will be doing some coding together
 - Sometimes you'll code and we then go through the solution together!
- You will have access to the slides throughout in case you need to go back to a particular slide

Here's What We'll Cover

1. Basic Coding in Python
2. Basics of Extracting and Manipulating Data
3. Troubleshooting Errors
4. How do I keep learning? (**Bonus**)

Prologue

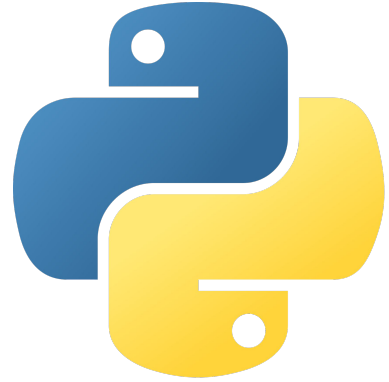
What is Python anyway?

What is it anyway?

Python is a **high-level programming** language

means it has strong amounts of abstraction,
which makes it understandable for humans

means it's writing code that give commands to
a computer



[Source](#)

History Lesson

It was built by Guido van Rossum in the 1980s

Python's popularity can be attributed to its:

- Small learning curve
- Strong developer community
- Adaptable with it's libraries



Where'd the name come from?



Monty *Python*

[Source](#)

Chapter 1

The Basics

This section is the longest

We need to cover:

1. Variables
2. Data Types
3. Operators
4. Conditionals
5. Loops

Variables

- The most basic building block in programming
- They are containers of data
- As the name indicates, the data you assign is *variable*
- You add data into a variable using the assignment operator “=”



variable = data

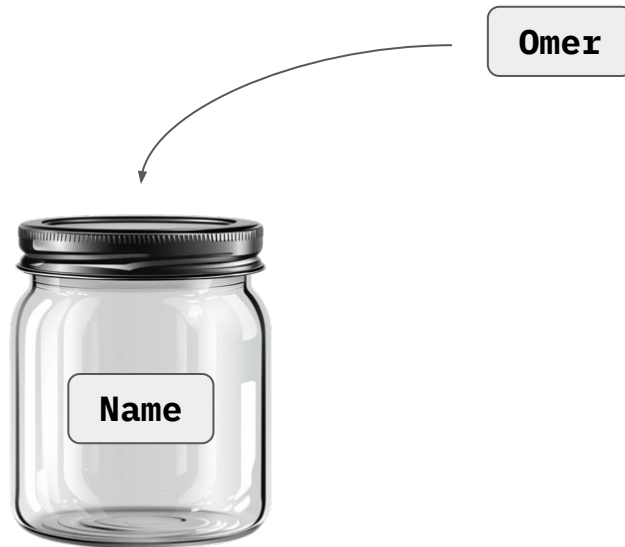
Variables

```
name = "Omer"
```

```
gpa = 3.2
```

```
siblings = 3
```

```
male = True
```



How do I name a variable?

- Names are case sensitive
- Names can only contain alphanumeric characters and underscores
- You cannot use keywords
 - These are reserved words for code that does specific processes (more on this shortly)
- Names cannot have spaces, start with numbers, or have a dash in them
- Descriptive names are always better

Camel Case

countOfTomatoes

Pascal Case

CountOfTomatoes

Snake Case

count_of_tomatoes

[Source](#)

Data Types

name = "Omer"

gpa = 3.2

siblings = 3

male = True

String: text data type

Float: anything with a decimal point

Int: whole numbers

Boolean: True or False

Arithmetic Operators

Used for mathematical operations

Symbol	Name	Example
+	Addition	$1+1 = 2$
-	Subtraction	$1-1 = 0$
/	Division	$6/2 = 3$
//	Floor division	$10//3 = 3$
*	Multiplication	$2*2 = 4$
%	Modulo	$5\%2 = 1$
**	Power	$3**2 = 9$

Comparison Operators

Used for comparing two values

Symbol	Name	Example (Result)
>	Greater than	3>1 (True)
<	Less than	3<1 (False)
!=	Not equals to	4!=2 (True)
==	Equals to	2==2 (True)
>=	Greater or equal to	3>=4 (False)
<=	Less or equal to	3<=3 (True)

Logical Operators

Used as part of conditional statements (will explain this more soon!)

Symbol	Name	Example (Result)
and	True if both statements are True	True and True (True)
or	True if either statements are True	False or False (False)
not	True if statement is False and vice versa	True (False)

Exercise time!

We are going to be using Jupyter Notebook for our coding needs today. It's an open-source web-application for writing computational documents. Let's get started!

- Open jupyter.org/try
- Click on JupyterLab
- Let's create a new notebook
 - Click on File -> New -> Notebook
 - Click next with the default settings
 - We're good to go!

HAVE SOME TEA



Let's take 5

Conditionals

- These are logical statements used to control the flow of code
- Keywords for conditionals are **if**, **elif**, and **else**
- This is how it looks

if (condition is True):

do this

elif (condition is True):

do this

else:

do this

Indentation is important! It indicates where an if/elif/else block ends

This is where the comparison and logical operators come in!

else catches everything *else*

Conditionals

```
money = 650
```

```
if (money > 1000):
```

```
    print("I have plenty for a car")
```

```
elif (money < 800 and money > 500):
```

```
    print("buy a scooter")
```

```
else:
```

```
    print("I need more money")
```

Exercise 1.1

I am making some Butter Chicken. We've already checked for onions but my mom's recipe recommends having at least two tomatoes. If I don't have tomatoes then, I can use 1.5 cans of tomato paste. Let's write some code that checks if we have these ingredients.



Exercise 1.2

I am organizing jars in my kitchen according to the amount of liquid they can hold. Small jars have a capacity of less than or equal to 250 ml. Medium jars have a capacity of more than 250 but less than 500 ml. Large jars have capacity of 500 ml or more.

I have a jar called “old jar” with a capacity of 440 ml. What size will it be?

You have **12 minutes** to solve this!



Loops

- Useful for iterating through code repeatedly
- There two main commands
 - `while()`:
 - Used when you need to keep running code until a condition is met
 - `for()`:
 - Used when you need iterate over set of variables/values

`while` (condition):
do something

`for` (instructions for iterating a set):
do something

Loops

```
#a rocket ship
```

```
countdown = 10
```

```
while (countdown > 0):
```

```
    print(countdown)
```

```
    countdown = countdown - 1
```

Infinite Loops

CAUTION

Remember to create a condition that ends a loop. If you don't do that, then a infinite loop may occur.



More keywords

There are keywords and functions that are useful in loops,

break - a keyword that will end the loop it's written in

continue - a keyword that will move the loop into the next iteration

in - an operator that checks if a variable is part of a set of variables

range() - a function that creates a list of integers based on the value in the parenthesis

```
for i in range(10):
```

```
    if i==8:
```

```
        break
```

```
    if i == 3:
```

```
        print("This is unlucky")
```

```
        continue
```

```
    else:
```

```
        print(i)
```

#the range(10) creates

Exercise 1.3

Time for the next step in making butter chicken: adding spices! Mom's recipe calls for 4 teaspoons of Garam Masala. Write a while loop that adds a teaspoon of Garam Masala until you reach the recommended amount.



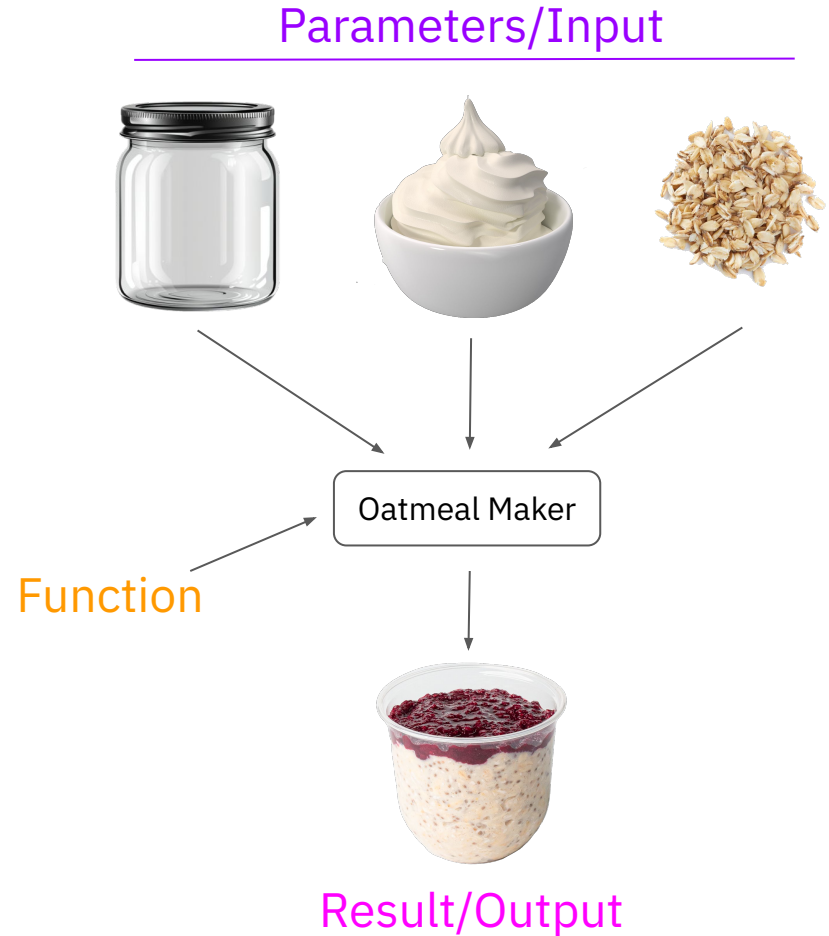
Functions

- A function is code which takes in parameters, performs tasks on them, and creates an output
- They only run when called
- Functions are useful for coding tasks that are repeated

For example, let's look at a function that displays variables onto your screen

```
print("Make me butter chicken")
```

Make me butter chicken



HAVE SOME TEA



Let's take 5

Chapter 2

Extracting and Manipulating Data

Data Structures

Before manipulating data, we need to understand data structures,

Data Structures are simply different ways of storing data

We will look at all built-in structures today

1. Lists
2. Tuples
3. Sets
4. Dictionaries

Lists and Tuples

- **Lists** and **Tuples** are an ordered sequence of elements separated by commas
- You can access these elements with their indexes (more on this shortly)
- The difference between them is that Tuples once created cannot be changed i.e. they are immutable
- Create lists with [] and tuples with ()
- Here's how it looks,

```
list_of_quantities = [11,2,3]    tuple_of_ingredients = ("onion", "yoghurt")
```

Sets

- The third data structure we'll be talking about today
- **sets** are unordered i.e. you can't refer to any particular element in the set
- No duplication is allowed
- Items in a set can be added or removed but each item cannot be changed
- Declared with {}
- Here's how it looks,

```
set_of_things = {"kiwi", 2, 10.5}
```

Dictionaries

- This data structure contains data as **key-value** pairs
- The key acts as the directory
- The value acts as well, the value
- Very much like a dictionary where the word is the key and the definition is the value
- Here's how it looks like to declare it,

```
contents = {  
    "Sodium" : "100 mg",  
    "Protein" : "9 g"  
}
```

The diagram illustrates the components of a dictionary declaration. A curved arrow labeled "key" points from the text "key" to the string "Sodium" in the code. Another curved arrow labeled "value" points from the text "value" to the string "100 mg" in the code.

Indexing

- How do we get the values in a list and/or dictionary?
- For lists, we use square brackets in front of the name of the list e.g. `list_of_quantitie[1]`
- Same applies for Tuples
- But for dictionaries, we have keys to find contents of the dictionary.
 - Hence we do something like this `contents["Sodium"]`

CAUTION

Indexes in all programming languages start with 0 and not 1. So the first element of a list will be at `list[0]`

Methods

- These structures have methods (think of it as a function) that help you do processes.
- `list.append()` adds an element to the end of a list
- `set.add()` adds an element to the set
- `list.reverse()` reverses the order of a list
- `dict.update()` adds a key-value pair

```
contents.update({"Fat": "4g"})
```

Data Structures

<i>Data Structure</i>	<i>Declared with</i>	<i>Ordered?</i>	<i>Changeable?</i>	<i>Duplicates?</i>
List	[]	Yes	Yes	Yes
Tuple	()	Yes	No	Yes
Set	{ }	No	No	No
Dictionary	{ : }	Yes	Yes	No

Exercise 2.1

I want to save my Mom's butter chicken recipe. I need you to save the steps in a list or dictionary.

Here are the steps:

Step 1: Add chopped onions

Step 2: Add chopped tomatoes

Step 3: Cook both vegetables with chicken



Exercise 2.2

I forgot to mention. There's another step,

Step 4: Toss in some garam masala

Add this to your data structure. (**Hint:** think about methods)

Chapter 3

Troubleshooting Errors

Common Errors

You're a human. Mistakes happen. And it's all good.

- Syntax Errors (your everyday typo)
- Index Errors (trying to look at a place that doesn't exist)
- Indentation Errors (you forgot to tab!)
- Name Errors (you're calling something that has a different name)
- Type Errors (trying to combine a string with an integer)

This is just a categorization. The actual error you see in the notebook will look different.

Find the error

```
if age > 20:
```

```
    print("Wow you're young")
```

```
if gpa = 3.2:
```

```
    print("my gpa is great!")
```

```
your_iq = 150
```

```
my_iq = "ten"
```

```
print(your_iq + my_iq)
```

```
list_of_ages = [12, 43, 32]
```

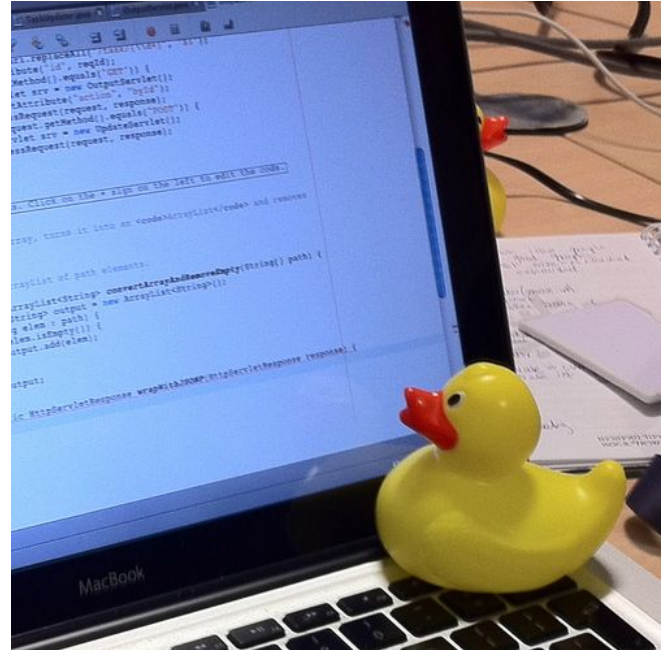
```
print(list_of_ages[4])
```

```
engagement_level = 8
```

```
print(engegment_leel)
```

How to deal with them

- Read the error!
- The best thing you can do is simply copy paste the error and Google it
- Look for answers in forums
 - Stack Overflow
 - Reddit
- If the code is complicated, then print() variables out to see if your code is behaving the way you want it to
- Explain the code to your rubber duckie!
- ChatGPT (be careful with this!)





Thank you for making butter chicken
with me!

Thank you for tuning in!

I am always available for a coffee chat. Drop me a message on LinkedIn (Scan the QR code).

I would love to hear some feedback on the workshop. Please fill out this [feedback survey](#) from the iSchool.

I'll see you in Python 102!



Citations

[Data Carpentry \(The Carpentries\)](#)

[Geeks for Geeks](#)

[W3schools](#)

[Freecodecamp](#)